

## **Лабораторная работа №2. Работа с дисплеем**

### **1. Цель работы**

Изучить механизм отправки и приёма текста и однобитных изображений для отображения их на OLED-дисплее робота-манипулятора и на холсте/текстуре 3D-модели робота в Unity.

### **2. Порядок выполнения**

- В проекте Unity организовать интерфейс передачи и отображения на экране текстовой строки:
  - кнопки отправки и приёма сообщения
  - камеру, смотрящую на экран робота, и окно вида с камеры
  - поле ввода строки для дальнейшей отправки на робота
- Передать текстовую строку [фамилия ИО], [группа] (всё латиницей) из Unity на робота и отобразить её на OLED-дисплее.
- Передать текстовую строку [фамилия ИО], [группа] (всё латиницей) из Arduino в Unity и отобразить на холсте (в виде текста) поверх 3D-модели дисплея.
- Передать bitmap-картинку (128x64) в формате байт-кода из Arduino в Unity, создать из неё текстуру и отобразить на 3D-модели дисплея.
- Продемонстрировать проект преподавателю, ответить на вопросы.
- Оформить отчёт и загрузить его в ЛК (в PDF).

### **3. Варианты задания**

Первая текстовая строка: Фамилия ИО (латиницей).

Вторая текстовая строка: номер группы.

Картинка может быть любым 1-битным изображением (состоящим только из чёрных или белых точек) размером 128x64 пикселя. Примеры можно найти по запросу "1-Bit Pixel Art".

#### 4. Сведения о дисплее

Робот-манипулятор Adept оснащается OLED-дисплеем типа SSD1306, подключаемый к контроллеру ATmega по интерфейсу I<sup>2</sup>C (рисунок 1). Дисплей имеет размер 0,96" и разрешение 128x64 пикселей. Частота работы – до 400 кГц. Кириллические символы не поддерживаются.



##### Распиновка:

- 1 – VDD – питание 5В
- 2 – GND – "земля"
- 3 – SCK – линия тактирование
- 4 – SDA – линия данных

**Рисунок 1. Дисплей SSD1306 и назначение его контактов**

На дисплей можно выводить не только текстовые символы, но также и однотонные изображения в формате bitmap. Изображения конвертируются в массив байтов и представляются в коде cpp, где 1 байт – это 8 пикселей. Соответственно, размер картинки для дисплея 128x64 будет равен 1 кБ. Например:

```
const unsigned char myBitmap[] PROGMEM = {  
    0x00, 0x00, 0x00, 0x00,  
    0xFF, 0x81, 0x81, 0xFF,  
    ...  
};
```

Для конвертации изображений в байт-код можно воспользоваться приложением:

<https://javl.github.io/image2cpp/>

I<sup>2</sup>C – это синхронная шина, использующая две двунаправленные последовательные линии связи: **SDA** (данные) и **SCL** (тактирование).

Устройство на шине может быть ведущим (master) или ведомым (slave). Инициатором обмена всегда выступает master, и обмен между двумя slave

невозможен. Классическая адресация включает 7-битное адресное пространство с 16 зарезервированными адресами – итого 112 адресов для подключения периферии. В основном режиме работы обмена происходит со скоростью 100 кбит/с (100 кГц).

## 5. Установка драйвера и Arduino IDE

Подключите робота к ПК кабелем microUSB – USB.

В данной модели робот используется преобразователь USB-TTL на чипе CH341 от китайской компании Jiangsu QinHeng (WCH). Он выступает в роли преобразователя интерфейсов и превращает порт USB в виртуальный COM-порт (последовательный порт, UART). Этот контроллер обычно используется для прошивки/отладки микроконтроллеров и других плат на микроконтроллерах, у которых нет собственного USB-порта. Также он применяется для связи с модемами, промышленным оборудованием и сетевыми устройствами, использующими последовательный интерфейс для управления и настройки.

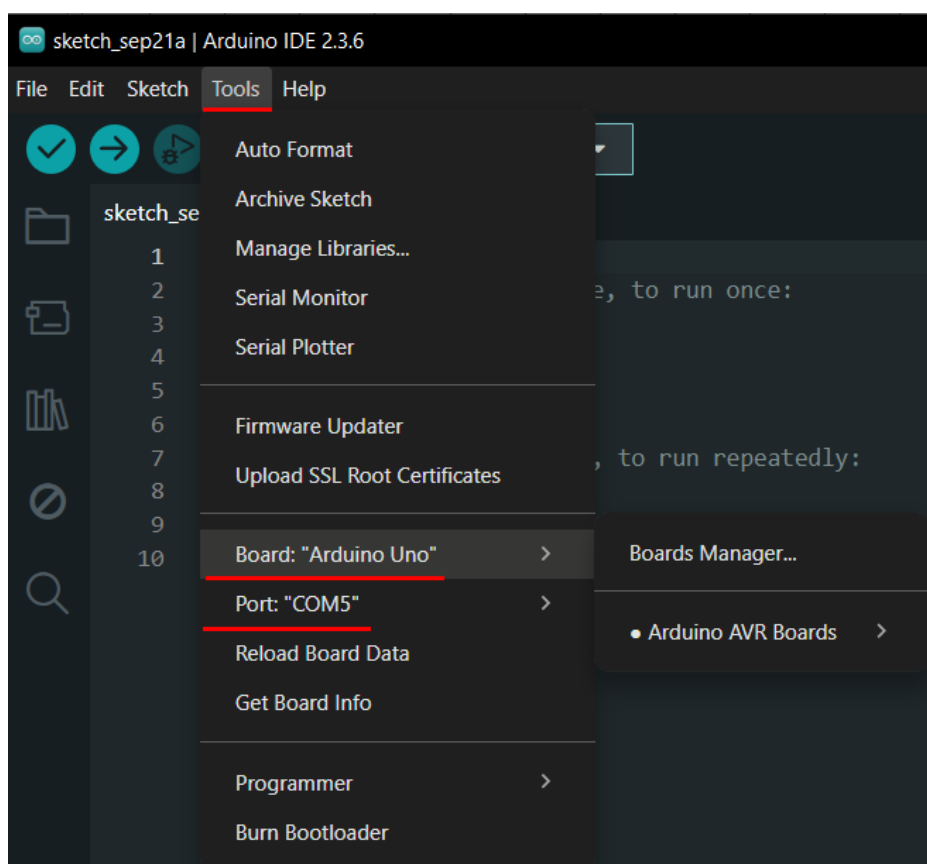


Рисунок 2. Подключение к контроллеру Arduino робота-манипулятора

Для работы CH341 нужно установить драйвер **CH341SER**:

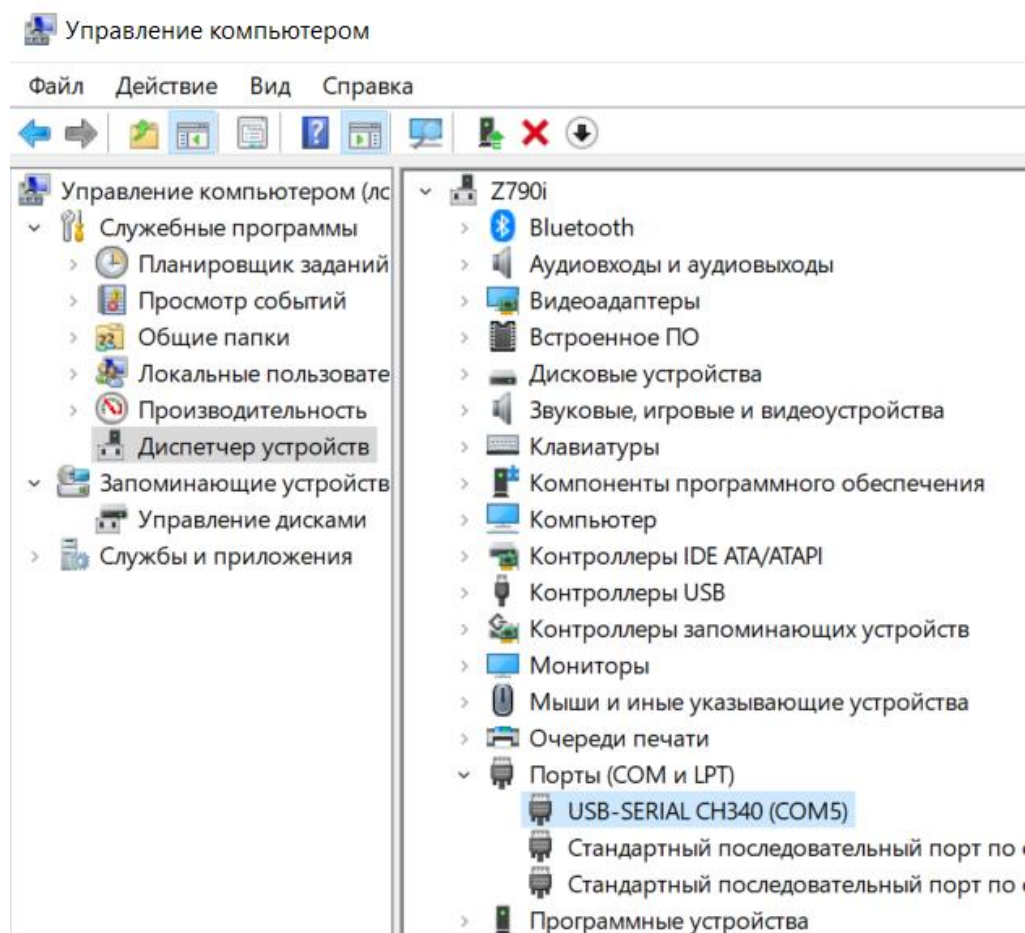
[https://www.wch-ic.com/downloads/ch341ser\\_exe.html](https://www.wch-ic.com/downloads/ch341ser_exe.html)

Далее перейдите на сайт разработчика Arduino:

<https://www.arduino.cc/en/software/#ide>

Скачайте и установите актуальную версию среды разработки Arduino IDE под свою операционную систему.

Запустите Arduino IDE. Там в главном меню **Tools** (инструменты) укажите номер виртуального COM-порта, который будет создан после установки драйвера для CH341, и версию Arduino – «**Arduino Uno**» (рисунок 2). Посмотреть номер COM-порта в Windows можно в окне «**Управление компьютером >> Диспетчер устройств >> Порты (COM и LPT)**» (рисунок 3).



**Рисунок 3. Виртуальный COM-порт в диспетчере устройств Windows**

Перед программированием манипулятора понадобится сконфигурировать директорию с библиотеками для Arduino. Вам понадобятся библиотеки

«Adafruit\_GFX\_Library», «Adafruit\_SSD1306» и «Adafruit\_BusIO». Скачать их можно с GitHub:

<https://github.com/adafruit/Adafruit-GFX-Library>

[https://github.com/adafruit/Adafruit\\_SSD1306](https://github.com/adafruit/Adafruit_SSD1306)

[https://github.com/adafruit/Adafruit\\_BusIO](https://github.com/adafruit/Adafruit_BusIO)

- **Adafruit-GFX-Library** – это графический движок, который предоставляет единый унифицированный набор функций для рисования базовых фигур, текста и изображений.
- **Adafruit\_SSD1306** – это библиотека для управления монохромными OLED-дисплеями с чипом-контроллером SSD1306. Представляет собой аппаратно-специфичный драйвер и используется для перевода общих команд от графического движка GFX в конкретные низкоуровневые команды и последовательности данных, понятные контроллеру дисплея SSD1306.
- **Adafruit\_BusIO** – это вспомогательная библиотека, которая предоставляет низкоуровневые функции для работы с аппаратными шинами I2C и SPI.

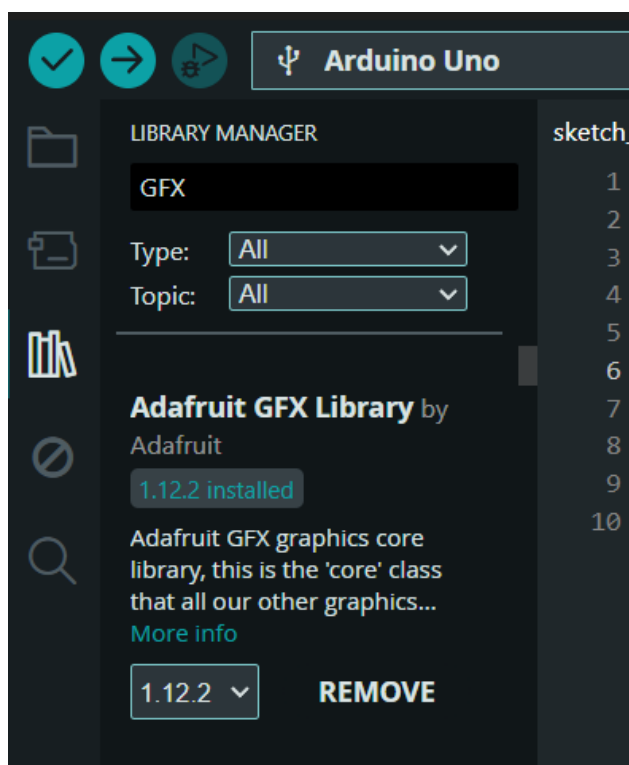


Рисунок 4. Установка библиотек в Arduino IDE

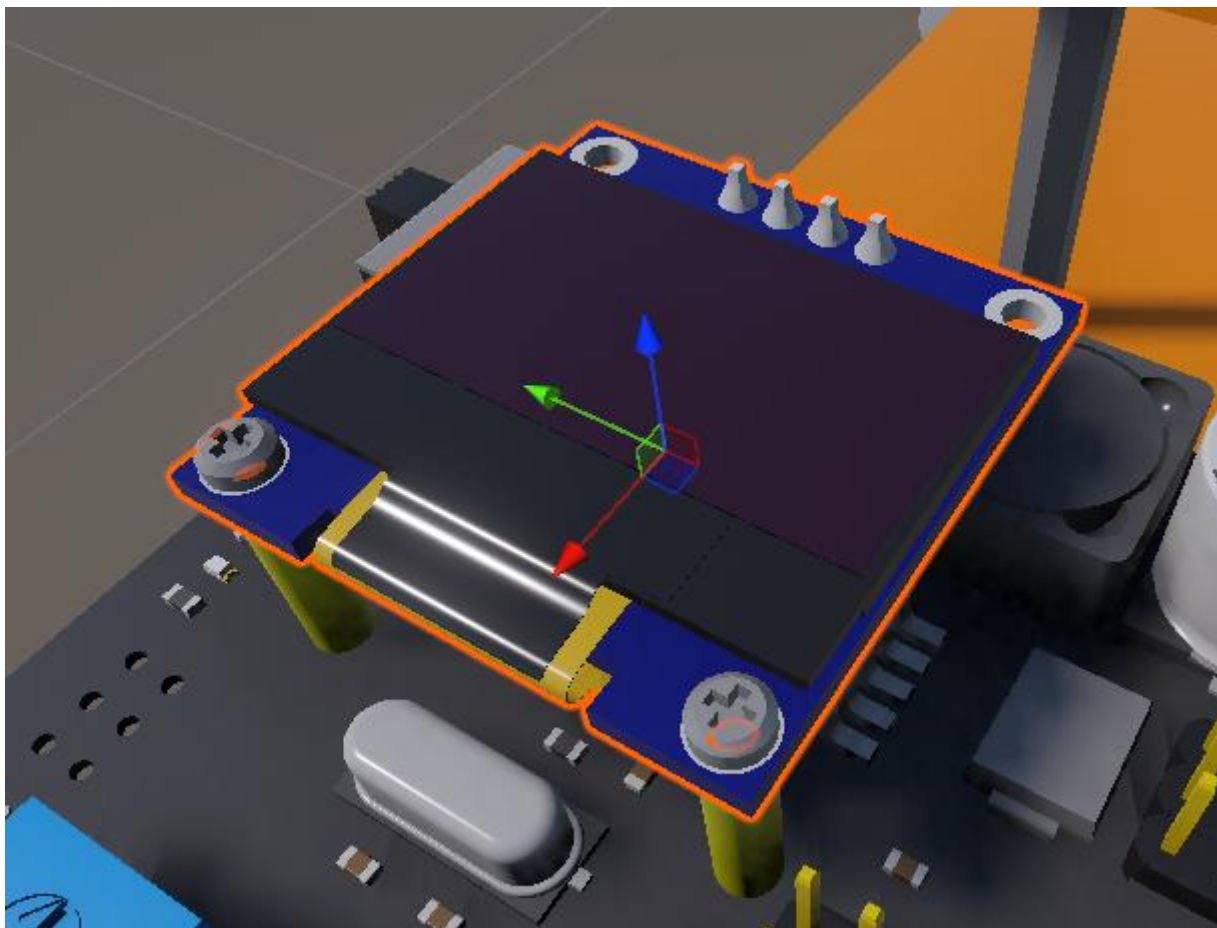
Скопируйте скачанные библиотеки в директорию «**libraries**». Если Arduino IDE установлена в C:\Program Files, то библиотеки будут храниться в каталоге пользователя:

C:\Users\[USERNAME]\Documents\Arduino\libraries

Либо библиотеки также можно установить через пункт бокового меню «Library Manager» внутри самой Arduino IDE (рисунок 4).

## 6. Работа с дисплеем

3D-модель «Adeert Arm» включает дисплей «OLED 0.96 Display» – он размещается над печатной платой и крепится к ней при помощи двух стоек M2x11 и винтов M2x5 (рисунок 5).



**Рисунок 5. 3D-модель дисплея SSD1306 в Unity**

Поскольку на дисплей через Arduino IDE можно выводить как текст, так и однобитовые изображения, нужно реализовать оба этих варианта в Unity.

## 6.1. Проверка дисплея в Arduino IDE

Для проверки работы оборудования производитель робота предоставляет тестовый скетч «Unpacking\_test\_code.ino». По умолчанию он расположен в прилагаемом архиве по пути:

../Adeept\_Robotic\_Arm\_Kit\Code\\_5 Unpacking\_test\_code\Unpacking\_test\_code

Откройте скетч в Arduino IDE. Выполните его проверку/компиляцию (Ctrl + R). В случае успешной компиляции загрузите скетч на плату (Ctrl + U).

Сразу после загрузки программы на OLED экране отобразится текст "OLED OK!". Если текст видно, значит дисплей исправен. Изменить отображаемый текст можно в строке

```
display.println(F("OLED OK"));
```

## 6.2. Инициализация и настройка дисплея

В коде скетча сперва выполняется подключение библиотек:

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

Далее следует инициализация параметров дисплея:

```
#define OLED_RESET -1
Adafruit_SSD1306 display(128, 64, &Wire, OLED_RESET);
```

Первая строка отвечает за выбор цифрового пина (D4) на Arduino, который будет отвечать за аппаратный сброс (Reset) контроллера дисплея. Но поскольку дисплей SSD1306 не имеет отдельного контакта для сброса, в коде пин сброса отключается.

Вторая строка – это конструктор. Она создает объект «display», через который выполняется управление дисплеем. В качестве параметров указывается разрешение, протокол связи (&Wire – стандартная библиотека Arduino для интерфейса I2C) и пин сброса.



Далее рассмотрим, как управлять дисплеем в методе `void setup()`.

```
display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

Эта команда физически обращается к дисплею и настраивает его электронику. В скобках константа `SSD1306_SWITCHCAPVCC` указывает дисплею использовать внутренние конденсаторы, чтобы поднять напряжение с 3,3 В или 5 В до необходимых ему для работы 7-9 В. Вторым параметром указывается адрес дисплея на шине I2C. Значение `0x3C` является наиболее распространённым.

```
display.setTextColor(WHITE); // Цвет текста
display.setCursor(0, 0); // Расположение курсора
display.setTextSize(2); // Размер текста
display.clearDisplay(); // Очистка дисплея
display.print("Строка"); // Вывод информации
display.display();
```

При вызове команды `display.display()` весь массив данных (1024 байта) за один раз передаётся в контроллер дисплея по шине I2C.

### 6.3. Вывод текста

Для вывода текста разместите поверх 3D-модели дисплея объект «Canvas» с режимом отображения «**World Space**» и на него добавьте элемент интерфейса «Text – Text Mesh Pro», рисунок 6.



Рисунок 6. Текст поверх дисплея в Unity



Дальнейшая логика довольно проста:

- 1) В Arduino IDE инициализировать дисплей и вывести на него строку текста:

```
char text[] = "robot"; // Строка для вывода

void setup() {
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    display.setTextColor(WHITE); //Задаёт цвет текста (WHITE / BLACK)
    display.clearDisplay(); // Очистка
}

display.print(text);
display.display(); // Запуск
```

- 2) Затем отправить строку по последовательному интерфейсу:

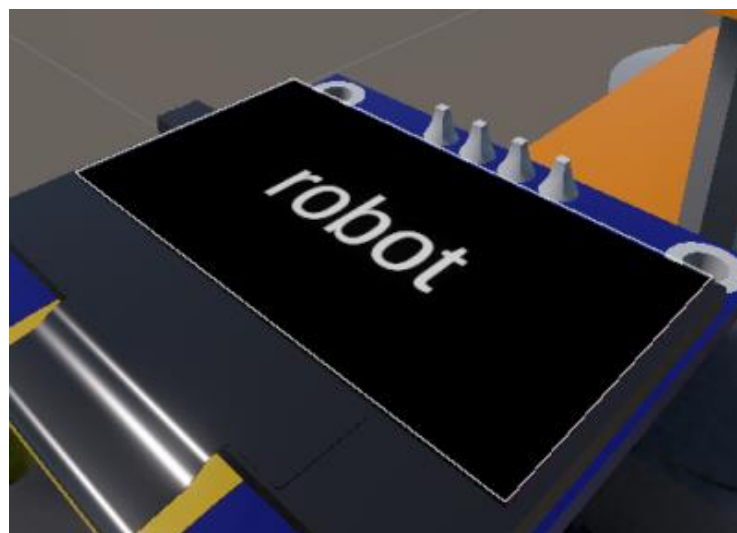
```
Serial.print(text); // Отправка по Serial
Serial.println(";");
delay(1000);
```

Не забудьте добавить задержку и символ конца строки для корректной работы парсера.

- 3) В Unity прочитать строку, записать данные в переменную outputLine и присвоить её значение текстовому элементу:

```
screenText.text = ExtractOledText(outputLine).ToString();
```

В результате на холсте должна появиться переданная строка (рисунок 7).



**Рисунок 7. Отображение строки текста на холсте дисплея**

Возникает вопрос, как передать в одной строке и углы поворота, и текст с экрана? Для этого можно выделить строку текста какими-нибудь спецсимволами, например, «%» и добавить в скрипт «GetSerialData.cs» ещё один парсер. Он должен получать строку, находить только текст между спецсимволами, записывать его в переменную, а всё остальное, включая сами спецсимволы удалять.

```
Serial.print("%");  
Serial.print(text); // Отправка по Serial  
Serial.print("%");  
Serial.println(";");  
delay(1000);
```

В Serial Monitor отобразится строка: **%robot%;**

Самый надёжный способ для работы с произвольными строками типа A1B2C3%ROBOT% – это использование метода `IndexOf`. Он найдёт первое и второе вхождение символа «%» и заберёт всё, что находится между ними.

```
int firstIndex = input.IndexOf('%'); // Индекс первого %...
```

Полный код парсера предлагается написать самостоятельно.

#### 6.4. Вывод картинки на OLED-дисплей

Микроконтроллер не понимает файлы картинок напрямую. Поэтому для отображения на OLED-дисплее картинки нужно сначала превратить её пиксели в массив байтов. Возьмите любое монохромное изображение размером 128x64 текселя и преобразуйте его в байт-код при помощи конвертера, например, **image2cpp** (рисунок 8). Для корректного отображения текстуры на OLED-дисплее выберите при конвертации в байт-код следующие параметры.

## 2. Image Settings

Canvas size(s):

picture.png (file resolution: 128 x 64)  
128 x 64 glyph

Background color:

☐ White ☒ Black ☐ Transparent

Invert image colors

☐

Brightness / alpha threshold:

128

0 - 255; if the brightness of a pixel is above the given level the pixel becomes white, otherwise they become black. When using alpha, opaque and transparent are used instead.

Scaling

original size

Center:

☐ horizontally ☐ vertically

Note: centering the image only works when using a canvas larger than the original image.

## 3. Preview



## 4. Output

Code output format

Arduino code

Adds some extra Arduino code around the output for easy copy-paste into [this example](#). If multiple images are loaded, generates a byte array for each and appends a counter to the identifier.

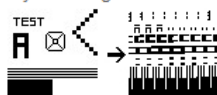
Identifier:

myBitmap

Draw mode:

Horizontal - 1 bit per pixel

If your image looks all messed up on your display, like the image below, try using a different mode.



### Рисунок 8. Параметры конвертации изображения в байт-код

- Размер холста (Canvas size): 128x64
- Цвет фона (Background color): Black (чёрный)
- Выходной формат (Output): Arduino code

Сгенерируйте байт-код и сохраните его в отдельном файле с расширением «.h» (например, **arduino\_image.h**). Положите этот файл рядом с файлом скетча Arduino IDE. Подключите библиотеки:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#include "arduino_image.h" // Файл с картинкой
```

В коде укажите:

```
display.drawBitmap(0, 0, myBitmap, 128, 64, WHITE); // Показать картинку
display.display(); // Отправить данные на дисплей
```

После компиляции кода и загрузки его в микроконтроллер на дисплее появляется картинка (рисунок 9).



Рисунок 9. Картинка bitmap и её отображение на OLED-дисплее

## 6.5. Передача изображения в Unity

Файл «arduino\_image.h» имеет вид:

```
#include <avr/pgmspace.h> // Нужно для работы PROGMEM
const unsigned char myBitmap [] PROGMEM = {
    0xaa, 0xaa, 0xaa, 0xaa, 0xaa, 0xaa, 0xaa, 0xaa, 0xaa, 0xaa, 0x7f,
    0xf2, 0xaa, 0xaa, 0xaa,
    ...
}
```

Как можно заметить, между символами { } располагается байт-код. Значит, если передать весь файл целиком по последовательному порту, то в Unity с помощью парсера несложно будет извлечь байты для того, чтобы превратить их в 2D-текстуру формата RGBA32.

Однако картинка в .h файле хранится как массив байтов в PROGMEM (флеш-памяти), отчего её нельзя передать одной командой как обычную строку. В

этом случае нужно использовать функцию **pgm\_read\_byte** для "доставания" из Flash-памяти одного байта или **memcpy\_P** для формирования буфера из 32 байт (в данном случае). При этом для передачи изображения из Flash-памяти сначала нужно считать его по частям, сохранить результат в RAM и уже затем отправить изображение целиком в Unity.

```
void sendImage() {
    const int bufferSize = 32;
    byte buffer[bufferSize];
    int totalBytes = 1024;

    for (int i = 0; i < totalBytes; i += bufferSize) {
        // Копирование блока из 32 байт из PROGMEM в RAM
        memcpy_P(buffer, &myBitmap[i], bufferSize);
        // Отправка буфера по последовательному порту
        Serial.write(buffer, bufferSize);
    }
}
```

Использование функций **pgm\_read\_byte** или **memcpy\_P** – это вынужденная мера, потому что у Arduino Uno всего 2 кБ оперативной памяти, а передаваемое изображение весит 1 кБ, т.е. занимает половину всей RAM. Ключевое слово **PROGMEM** в файле изображения «**arduino\_image.h**» переводит массив во Flash-память, объём которой составляет 32 кБ. Там же хранится и код программы. Но микроконтроллер ATmega не может просто так прочесть данные из Flash-памяти, поэтому используется специальная буферная команда, которая говорит процессору, что данные по указанному адресу нужно искать не в RAM, а во Flash.

**memcpy\_P** – это оптимизированная ассемблерная функция, которая максимально быстро перемещает 32 байта данных во временный буфер в RAM. Её использование обосновано тем, что библиотека **Serial** гораздо быстрее обработает один большой фрагмент объёмом 32 кБ, чем 32 маленьких фрагмента по 1 кБ.

Если сейчас открыть Serial Monitor, там будет виден "шум" из квадратов и нечитаемых символов. Это нормальное состояние, поскольку данные передаются в бинарном, а не в текстовом виде:



Unity умеет создавать текстуры из массивов байтов. Однако для работы этой функции важно учесть различия в форматах данных. Unity использует форматы RGB24 или RGBA32 и не умеет обрабатывать однобитные текстуры без конвертации. Требуется, чтобы на 1 канал приходилось минимум 8 бит. Формат данных SSD1306, напротив, предполагает, что один бит – это один пиксель. Поэтому понадобится написать парсер и конвертер форматов.

Пусть скрипт называется «**CreateSerialTexture.cs**». Он будет выглядеть следующим образом. В методе `Start()` открывается порт и задаются параметры текстуры. Важно отключить фильтрацию и тайлинг, чтобы текстура не размывалась и не повторялась.

```
void Start()
{
    sp.Open(); // Открыть порт
    texture = new Texture2D(width, height, TextureFormat.RGBA32, false);
    texture.filterMode = FilterMode.Point;
    texture.wrapMode = TextureWrapMode.Clamp;
}
```

Далее в методе `Update()` выполняется чтение принятого массива и создание буфера размером 128x64.

```
void Update()
{
    if (sp.BytesToRead >= (width * height / 8))
    {
        byte[] buffer = new byte[width * height / 8];
        sp.Read(buffer, 0, buffer.Length);
        CreateTexture(buffer);
    }
}
```

Библиотека `Adafruit_GFX` использует довольно простой формат представления данных: она упаковывает пиксели строго по горизонтали, где 1 байт = 8 пикселей в ряд. Самый левый пиксель – это старший бит (0x00).

- Первый байт в .h файле – это пиксели с 0 по 7 в самой верхней строке (Y=0).
- Второй байт в .h файле – это пиксели с 8 по 15 в той же строке.

- Так до конца строки, после чего идёт переход на следующую строку (Y=1).

Но поскольку в Unity координаты UV текстуры начинаются снизу слева, то для правильной ориентации понадобится также инвертировать ось Y. Метод `CreateTexture` будет отвечать за создание текстуры из принятых битов.

```
void CreateTexture(byte[] data)
{
    int byteIndex = 0;
    for (int y = 0; y < height; y++)
    {
        for (int x = 0; x < width; x += 8)
        {
            ...
        }
    }
}
```

Его предлагается написать самостоятельно.

## 6.6. Отображение картинки в Unity

Добавьте поверх 3D-модели дисплея (игровой объект «OLED 0.96 Display») квад (Quad Mesh) так, чтобы он соответствовал размеру и положению OLED-матрицы. Сделайте его дочерним для дисплея. Примените к нему материал типа `Unlit/Texture`, чтобы картинка была хорошо видна вне зависимости от освещения. Примените скрипт «`CreateSerialTexture.cs`» к кваду. Никаких настраиваемых параметров у него нет. Сразу после запуска игры должен открыться COM-порт, после чего скрипт считывает биты, сформирует из них картинку и применит её к материалу квада. Вид полученной текстуры показан на рисунке 10.



**Рисунок 10. Вид картинки на модели дисплея SSD1306 в Unity**



## 7. Интерфейс для дисплея

Добавьте на экран отдельную кнопку, которая будет вызывать метод чтения строки и вывода её на холст дисплея. Порядок действий тут следующий:

- Временно деактивировать кнопку очистки данных (`clearButton`);
- Прочитать строку, пришедшую с последовательного порта (`ReadLine()`);
- Передать эту строку в парсер, чтобы он оставил только текст между спецсимволами (`ExtractOledText(outputLine)`);
- Выходную строку из парсера передать в переменную `screenText.text`;
- Снова активировать кнопку очистки данных;

Также для наглядности будет нелишним добавить вторую камеру (орфографическую), смотрящую прямо на OLED дисплей. Вывод с этой камеры можно разместить рядом с кнопкой считывания строки. Для этого создайте текстуру «Render Texture», перетащите её в поле «Output Texture» второй камеры. Создайте компонент «UI >> Raw Image» на холсте GUI, и укажите ссылку на «Render Texture» в поле «Texture». Не забудьте удалить компонент «Audio Listener» у второй камеры во избежание конфликтов.

Пример полученного интерфейса приведён на рисунке 11.

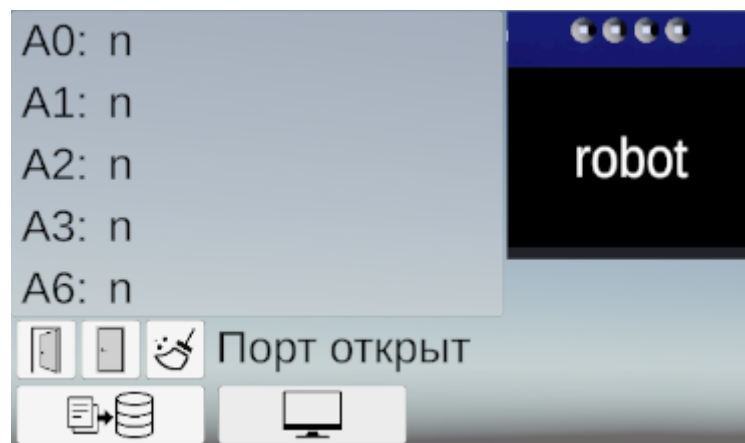


Рисунок 11. Интерфейс работы с дисплеем

## 8. Отправка текста из Unity на OLED-дисплей

Добавьте в сцену Unity интерфейс для отправки текста (рисунок 12):

- поле ввода (Input Field)
- кнопку отправки (Button)

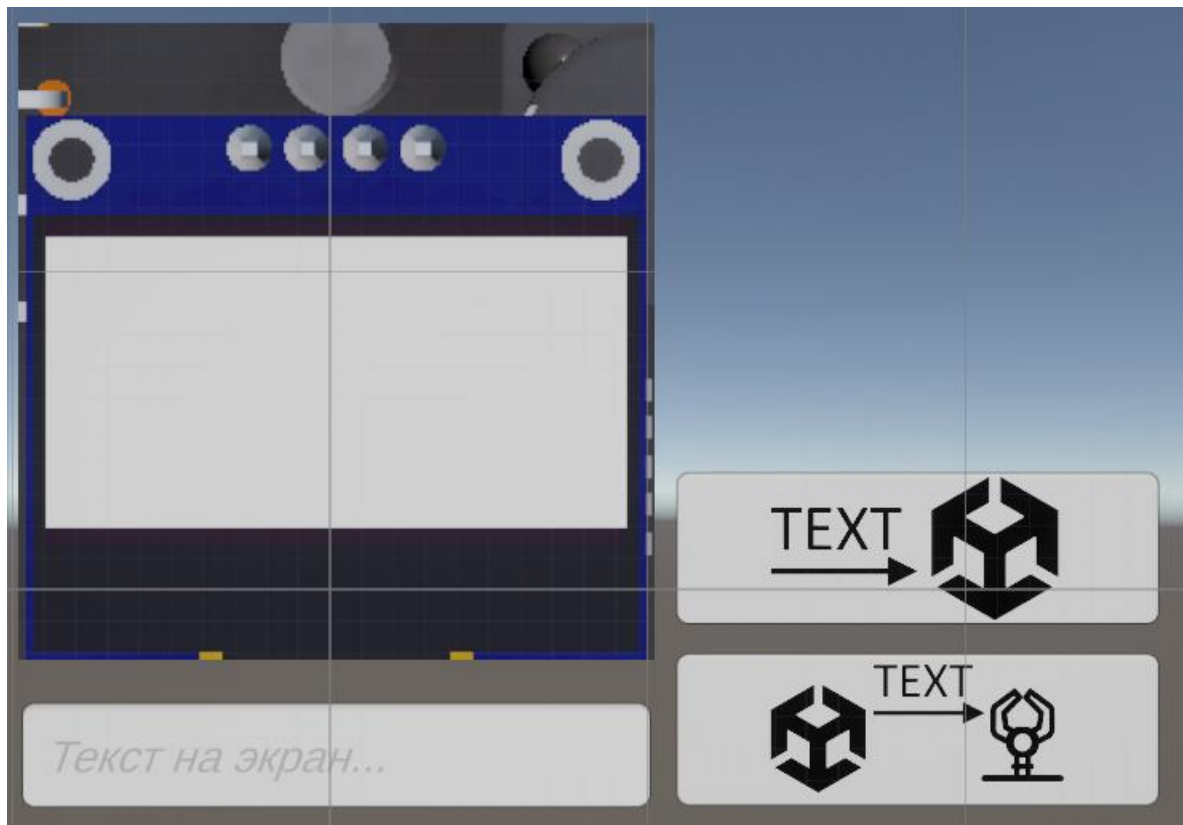


Рисунок 12. Интерфейс для отправки текста

Логика работы простая: при нажатии на кнопку отправки текст, находящийся в данный момент в поле ввода (**InputField (TMP) >> Text Area >> Text**) должен отправиться в виде строки по последовательному интерфейсу на робота.

Добавьте в Unity в скрипт, ответственный за работу с Serial-портом, метод для отправки текста:

```
public void SendTextToOled()
{
    if (textInField != "")
    {
        clearButton.interactable = false;
        serialPort.WriteLine(textInField);
        Debug.Log("Отправлена строка: " + textInField);
        clearButton.interactable = true;
    }
}
```

```
}
```

Введённый в поле UnputField текст должен также появляться на модели дисплея в Unity.

Теперь надо принять отправленный текст на стороне робота и вывести строку на экран:

```
display.print(text);
```

Обратите внимание, что скорость работы Serial-порта, объявленная в скетче Arduino, должна совпадать со скоростью, обозначенной в скрипте Unity! Также не забудьте обозначить символ конца строки (;), чтобы контроллер понимал, когда заканчивать чтение, и обрезать его, чтобы он не выводился на дисплей.

```
void setup() {  
    Serial.begin(9600);  
}  
  
void loop() {  
    // Чтение строки до символа новой строки  
    String textFromUnity = Serial.readString(';');  
    textFromUnity.trim();  
    ...  
}
```

Написать код обработки строки и вывода её на OLED предлагается самостоятельно. Результат показан на рисунке 13.



**Рисунок 13. Передача сообщения на робота**

## 9. Содержание и оформление отчёта

Отчёт о лабораторной работе оформляется в соответствии с требованиями ГОСТ 7.32 – 2017, которые доступны по адресу: <https://guap.ru/regdocs/docs/uch>

Содержание отчёта о лабораторной работе:

- 1) Титульный лист.
- 2) Цель работы.
- 3) Вариант задания: текстовые строки и картинка.
- 4) Алгоритм, краткое описание, листинг кода:
  - Как передать текст из Arduino в Unity.
  - Как передать текст из Unity в Arduino.
  - Как передать по Serial-интерфейсу bitmap-изображение.
  - Как принять бинарные данные в Unity и превратить их в текстуру.
- 5) Итоговые фото и скриншоты результатов.
- 6) Выводы о проделанной работе.